

Guide to Typst

09.03.2026 - v1.3.0

for [typst v0.14.2](#)

for [hei-synd-thesis v0.3.0](#)

Silvan Zahno

silvan.zahno@hevs.ch

HEI-Vs

Contents

1	Introduction	4
2	Installation	5
2.1	With cargo	5
2.2	MacOS	5
2.3	Linux	5
2.4	Windows	5
3	Formatting	6
3.1	Markup	6
3.2	Page Formatting	6
3.3	Space	6
3.4	Text Formatting	6
4	Elements	10
4.1	Headings	10
4.2	Lists	10
4.3	Custom Lists	11
4.4	Minitoc	12
4.5	Images	13
4.5.1	Alignment	13
4.5.2	Caption	13
4.5.3	Cluster	13
4.6	Tables	16
4.6.1	Karnaugh Tables	19
4.7	Icon Boxes	21
4.8	Color Boxes	22
4.8.1	Todo Box	23
4.8.2	Option Style	23
4.9	Title Box	24
4.10	Exam Header	24
4.11	Exam Reminder	26
4.12	Exercise Message	28
4.13	Word/Character Count	29

5	References	30
5.1	Links	30
5.2	Crossreferences	30
5.3	External References	30
5.4	Glossary	31

Figures

Figure 1	One image one caption	13
Figure 2	Caption left image	14
Figure 3	Caption right image	14
Figure 4	Caption topleft image	15
Figure 5	Caption topright image	15
Figure 6	Caption bottomleft image	15
Figure 7	Caption bottomright image	15
Equation 8	Some proof	33

Tables

Table 1	Multiple images one caption	13
Table 2	Multiple images one caption	14
Table 3	Table caption	16
Table 4		19
Table 5		19
Table 6		19
Table 7		20
Table 8	Links	30

Listings

Listing 1	Label inserts	30
Listing 2	Rust Code	32

Equations

Equation (1)		33
Equation (2)		33
Equation (3)		33

Equation (4) 33

Equation (5) 34

Equation (6) 34

Equation (7) 34

Equation (8) 34

Equation (9) 35

Equation (10) 35

Equation (11) 35

Equation (12) 35

Equation (13) 35

Equation (14) 35

Equation (15) 35

Equation (16) 35

Equation (17) 35

Equation (18) 35

Equation (19) 35

Equation (20) 35

Equation (21) 35

Equation (22) 35

Equation (23) 35

1 | Introduction

The goal of this document is to have the most common used elements for the markup language **typst** readily available. A detailed documentation can be found on their website: <https://typst.app/docs>

It is to note that these are most common used elements for the template and library [hei-synd-thesis](#). This template includes the following additional libraries:

- [codelst](#) - Source code formatting
- [codly](#) - Source code formatting
- [glossarium](#) - Glossary functionality
- [wordometer](#) - Word counts and document statistics
- [icu-datetime](#) - Date and time formatting
- [cheq](#) - Markdown-like checklists
- [muchpdf](#) - include PDF images
- all files can be found in the [github repo](#) such as
 - **boxes.typ**
 - **constants.typ**
 - **helpers.typ**
 - **il8n-thesis.json**
 - **items.typ**
 - **pages-*.typ**
 - **template-*.typ**

In order to use the [hei-synd-thesis](#) as library import it as follows:

```
1 #import "@preview/hei-synd-thesis:0.3.0": *
```

2 | Installation

2.1 With cargo

If you use already the **rust** programming language then you can use rust to install the latest toolchain.

```
1 # install rust and cargo
2 curl https://sh.rustup.rs -sSf | sh
3
4 # install typst
5 cargo install --git https://github.com/typst/typst
```

2.2 MacOS

On MacOS you can use **homebrew**

```
1 # install homebrew
2 /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/
3 install.sh)"
4 # install typst
5 brew install typst
```

2.3 Linux

In Linux you can use the commonly available package manager

```
1 brew install typst
2 pacman -S typst
3 xbps-install typst
4 sudo apt-get install typst
```

2.4 Windows

On Windows you can use **chocolatey**. See: <https://chocolatey.org/install>

```
1 # install chocolatey
2 # ensure to use a administrative powershell
3 Set-ExecutionPolicy Bypass -Scope Process -Force;
4 [System.Net.ServicePointManager]::SecurityProtocol =
5 [System.Net.ServicePointManager]::SecurityProtocol -bor 3072; iex ((New-Object
6 System.Net.WebClient).DownloadString('https://community.chocolatey.org/install.ps
7 1'))
8
9 # install typst
10 choco install typst
```

3 | Formatting

3.1 Markup

Name	Example	Raw
Singleline Comment		//
Multiline Comment		/* */
Paragraph break		blankline
Line break		\
bold	bold	*bold*
italic	<i>italic</i>	_italic_
monospaced	monospaced	`monospaced`
math	$x = 1$	\$x=1\$
lowercase	lower	#lower("LoWeR")
uppercase	UPPER	#upper("UpPeR")
smallcaps	SMALLCAPS	#smallcaps("SmallCaps")
smartquote	“test”	#smartquote()test#smartquote()
overline	<u>overline</u>	#overline("overline")
underline	<u>underline</u>	#underline("underline")
strike	strike	#strike("strike")
sub	Text _{sub}	Text#sub("sub")
super	Text ^{super}	Text#super("super")
Label		<label>
Reference		@label

3.2 Page Formatting

```
1 #pagebreak() // pagebreak
2 #parbreak() // parbreak
3 \ // linebreak
```

3.3 Space

A

B

```
1 A #h(5cm) B,
```

C

D

```
1 C #v(0.2cm) D
```

3.4 Text Formatting

Custom textsizes and colors are available in the [constants.typ](#):

Name	Example	Raw
Sizes	8pt text tiny text	<pre>1 text(8pt, "8pt text") 2 text(tiny "tiny text")</pre>
	9pt text smaller text	<pre>1 text(9pt, "9pt text") 2 text(smaller "smaller text")</pre>
	10pt text small text	<pre>1 text(10pt, "10pt text") 2 text(small "small text")</pre>
	11pt text normal text	<pre>1 text(11pt, "11pt text") 2 text(normal "normal text")</pre>
	14pt text large text	<pre>1 text(14pt, "14pt text") 2 text(large "large text")</pre>
	16pt text larger text	<pre>1 text(16pt, "16pt text") 2 text(larger "larger text")</pre>
	24pt text huge text	<pre>1 text(24pt, "24pt text") 2 text(huge "huge text")</pre>
Types	36pt text huger text	<pre>1 text(36pt, "36pt text") 2 text(huger "huger text")</pre>
	Fira Sans	<pre>1 text(font:"Fira Sans", "Fira Sans")</pre>
	Fira Mono	<pre>1 text(font:"Fira Mono", "Fira Mono")</pre>
	Source Sans Pro	<pre>1 text(font:"Source Sans Pro", "Source Sans Pro")</pre>
	Arial	<pre>1 text(font:"Arial", "Arial")</pre>

Name	Example	Raw
	Times New Roman	<code>text(font:"Times New Roman", "Times New Roman")</code>
Alignment	start	<code>align(start){start}</code>
	end	<code>align(end){end}</code>
	left	<code>align(left){left}</code>
	center	<code>align(center){center}</code>
	right	<code>align(right){right}</code>
	top	<code>align(top){top}</code>
	horizon	<code>align(horizon){horizon}</code>
	bottom	<code>align(bottom){bottom}</code>
	center + horizon	<code>align(center + horizon){center + horizon}</code>
Colors	black	<code>#text(fill:black)[black]</code>
	red	<code>#text(fill:red)[red]</code>
	green	<code>#text(fill:green)[green]</code>
	blue	<code>#text(fill:blue)[blue]</code>
	purple	<code>#text(fill:purple)[purple]</code>
	gray-80	<code>#text(fill:gray-80)[gray-80]</code>
	gray-70	<code>#text(fill:gray-70)[gray-70]</code>
	gray-60	<code>#text(fill:gray-60)[gray-60]</code>
	gray-50	<code>#text(fill:gray-50)[gray-50]</code>
	gray-40	<code>#text(fill:gray-40)[gray-40]</code>
	gray-30	<code>#text(fill:gray-30)[gray-30]</code>
	gray-20	<code>#text(fill:gray-20)[gray-20]</code>
	gray-10	<code>#text(fill:gray-10)[gray-10]</code>
	hei-orange	<code>#text(fill:hei-orange)[hei-orange]</code>
	hei-blue	<code>#text(fill:hei-blue)[hei-blue]</code>
	hei-pink	<code>#text(fill:hei-pink)[hei-pink]</code>
	hei-yellow	<code>#text(fill:hei-yellow)[hei-yellow]</code>
	hei-green	<code>#text(fill:hei-green)[hei-green]</code>
	spl-green	<code>#text(fill:spl-green)[spl-green]</code>
	spl-blue	<code>#text(fill:spl-blue)[spl-blue]</code>
	spl-pink	<code>#text(fill:spl-pink)[spl-green]</code>
	color-info	<code>#text(fill:color-info)[color-info]</code>
	color-idea	<code>#text(fill:color-idea)[color-idea]</code>

Name	Example	Raw
	color-warning	<code>#text(fill:color-warning)[color-warning]</code>
	color-important	<code>#text(fill:color-important)[color-important]</code>
	color-fire	<code>#text(fill:color-fire)[color-fire]</code>
	color-rocket	<code>#text(fill:color-rocket)[color-rocket]</code>
	color-todo	<code>#text(fill:color-todo)[color-todo]</code>
	code-bg	<code>#text(fill:code-bg)[code-bg]</code>
	code-border	<code>#text(fill:code-border)[code-border]</code>

4 | Elements

Contents

4.1	Headings	10
4.2	Lists	10
4.3	Custom Lists	11
4.4	Minitoc	12
4.5	Images	13
4.5.1	Alignment	13
4.5.2	Caption	13
4.5.3	Cluster	13
4.6	Tables	16
4.6.1	Karnaugh Tables	19
4.7	Icon Boxes	21
4.8	Color Boxes	22
4.8.1	Todo Box	23
4.8.2	Option Style	23
4.9	Title Box	24
4.10	Exam Header	24
4.11	Exam Reminder	26
4.12	Exercise Message	28
4.13	Word/Character Count	29

4.1 Headings

```
1 = Heading 1
2 == Heading 1.1
3 === Heading 1.1.1
4 ==== Heading 1.1.1.1
5 ...
```

4.2 Lists

- First
- Second
- Third

```
1 - First
2 - Second
3 - Third
```

- First
 - Second
 - Third

```
1 - First
2 - Second
3 - Third
```

- First
- Second
- Third

```
1 - First
2 - Second
3 - Third
```

- First
- Second
- Third

```
1 list(
2   [First],
3   [Second],
4   [Third],
5 )
```

1. First
 1. Second
2. Third

```
1 + First
2 + Second
3 + Third
4 Text
5 4. Fourth
6 + Fifth
```

Text

4. Fourth
5. Fifth

1. First
 - a) Second
2. Third

```
1 + First
2 #set enum(numbering: "a")
3 + Second
4 + Third
5 Text
6 4. Fourth
7 + Fifth
```

Text

4. Fourth
5. Fifth

4.3 Custom Lists

Custom lists are available in the [items.typ](#):

- ☑ item-list
- 🔖 item-checkbadge
- ◯ item-circle
- ◻ item-square
- ☑ item-checkcircle
- ☑ item-checksquare
- ✓ item-check
- 📁 item-file
- 📁 item-folder
- ⊗ item-xcircle
- ⊗ item-xsquare
- ✕ item-x

```
1 #item-list()[item-list]
2 #item-checkbadge()[item-checkbadge]
3 #item-circle()[item-circle]
4 #item-square()[item-square]
5 #item-checkcircle()[item-checkcircle]
6 #item-checksquare()[item-checksquare]
7 #item-check()[item-check]
8 #item-file()[item-file]
9 #item-folder()[item-folder]
10 #item-xcircle()[item-xcircle]
11 #item-xsquare()[item-xsquare]
12 #item-x()[item-x]
```

4.4 Minitoc

The `minitoc` is specific for this template and allows to show a TOC between two labels. The `minitoc` is also used within the `#add-chapter` function if `after` and `before` labels are defined.

Contents

4.1	Headings	10
4.2	Lists	10
4.3	Custom Lists	11
4.4	Minitoc	12
4.5	Images	13
4.5.1	Alignment	13
4.5.2	Caption	13
4.5.3	Cluster	13
4.6	Tables	16
4.6.1	Karnaugh Tables	19
4.7	Icon Boxes	21
4.8	Color Boxes	22
4.8.1	Todo Box	23
4.8.2	Option Style	23
4.9	Title Box	24
4.10	Exam Header	24
4.11	Exam Reminder	26
4.12	Exercise Message	28
4.13	Word/Character Count	29

```
1 #minitoc(after: <sec:elem>, before: <sec:ref>)

1 #add-chapter(
2   "/02-main/03-elements.typ",
3   after: <sec:elem>,
4   before: <sec:ref>,
5 )
6 // or
7 #add-chapter(
8   after: <sec:elem>,
9   before: <sec:ref>,
10 ) [
11   Content of the Chapter
12 ]
```

4.5 Images

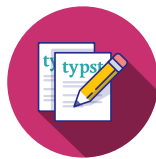
4.5.1 Alignment

left



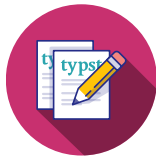
```
1 #image("/04-resources/icon.svg",
2   width: 2cm)
```

center



```
1 #align(center,
2   image("/04-resources/icon.svg",
3     width: 2cm)
4 )
```

right



```
1 #align(right,
2   image("/04-resources/icon.svg",
3     width: 2cm)
4 )
```

4.5.2 Caption

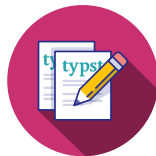


Figure 1 - One image one caption

```
1 #figure(
2   image("/04-resources/icon.svg",
3     width: 2cm),
4   caption: [One image one caption]
5 ) <fig:icon>
```

4.5.3 Cluster

Two images one caption

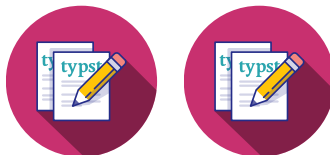


Table 1 - Multiple images **one** caption

```
1 #figure(
2   table(
3     columns: 2,
```

```

4     stroke: none,
5     align: center + horizon,
6     image(icon, width: 2cm), image(icon, width: 2cm)
7   ),
8   caption: [Multiple images *one* caption]
9 )

```

Four images one caption

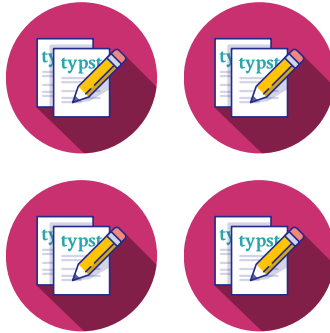


Table 2 - Multiple images **one** caption

```

1 #figure(
2   table(
3     columns: 2,
4     stroke: none,
5     align: center + horizon,
6     image(icon, width: 2cm), image(icon, width: 2cm),
7     image(icon, width: 2cm), image(icon, width: 2cm),
8   ),
9   caption: [Multiple images *one* caption]
10 )

```

Two images two caption



Figure 2 - Caption left image Figure 3 - Caption right image

```

1 #align(center,
2   table(
3     columns: 2,
4     stroke: none,
5     align: center + horizon,
6     figure(image(icon, width: 2cm), caption: [Caption left image]),
7     figure(image(icon, width: 2cm), caption: [Caption right image]),
8   ))

```

Four images four caption



Figure 4 - Caption topleft image



Figure 5 - Caption topright image



Figure 6 - Caption bottomleft image



Figure 7 - Caption bottomright image

```

1 #align(center,
2   table(
3     columns: 2,
4     stroke: none,
5     align: center + horizon,
6     figure(image(icon, width: 2cm), caption: [Caption topleft image]),
7     figure(image(icon, width: 2cm), caption: [Caption topright image]),
8     figure(image(icon, width: 2cm), caption: [Caption bottomleft image]),
9     figure(image(icon, width: 2cm), caption: [Caption bottomright image]),
10  ))

```

4.6 Tables

Tables with and without caption

	Col1	Col2
Row1	cell-0-0	cell-1-0
Row2	cell-0-1	cell-1-1

	Col1	Col2
Row1	cell-0-0	cell-1-0
Row2	cell-0-1	cell-1-1

Table 3 - Table caption

```
1 #table(  
2   columns: 3,  
3   align: center + horizon,  
4   table.header([], [*Col1*] ,  
5   [*Col2*]),),  
6   [*Row1*], "cell-0-0", "cell-1-0",  
7   [*Row2*], "cell-0-1", "cell-1-1",  
8 )
```

```
1 #figure(  
2   table(  
3     columns: 3,  
4     align: center + horizon,  
5     table.header([], [*Col1*] ,  
6     [*Col2*]),),  
7     [*Row1*], "cell-0-0", "cell-1-0",  
8     [*Row2*], "cell-0-1", "cell-1-1",  
9   ),  
10  kind: table,  
11  caption: [Table Caption]
```

Tables with cell spans

	Col1	Col2
Row1	cell-0	cell-1-0
Row2		cell-1-1

	Col1	Col2
Row1	cell-0	
Row2	cell-0-1	cell-1-1

```
1 #table(  
2   columns: 3,  
3   align: center + horizon,  
4   table.header([], [*Col1*] ,  
5   [*Col2*]),),  
6   [*Row1*], table.cell(rowspan: 2)  
7   [cell-0], "cell-1-0",  
8   [*Row2*],  
9   "cell-1-1",  
10 )
```

```
1 #table(  
2   columns: 3,  
3   align: center + horizon,  
4   table.header([], [*Col1*] ,  
5   [*Col2*]),),  
6   [*Row1*], table.cell(colspan: 2)  
7   [cell-0],  
8   [*Row2*], "cell-0-1", "cell-1-1",  
9 )
```

Table Design

	Col1	Col2
Row1	cell-0-0	cell-1-0
Row2	cell-0-1	cell-1-1

	Col1	Col2
Row1	cell-0-0	cell-1-0
Row2	cell-0-1	cell-1-1

```
1 #table(  
2   columns: 3,
```

```
1 #table(  
2   columns: 3,
```

```

3 align: center + horizon,
4 table.header([], [*Col1*] ,
5 [*Col2*]),),
6 [*Row1*], "cell-0-0", "cell-1-0",
7 [*Row2*], "cell-0-1", "cell-1-1",
8 )

```

	Col1	Col2
Row1	cell-0-0	cell-1-0
Row2	cell-0-1	cell-1-1

```

3 align: center + horizon,
4 table.header([], [*Col1*] ,
5 [*Col2*]),),
6 [*Row1*], "cell-0-0", "cell-1-0",
7 [*Row2*], "cell-0-1", "cell-1-1",
8 )

```

	Col1	Col2
Row1	cell-0-0	cell-1-0
Row2	cell-0-1	cell-1-1

```

1 #table(
2   columns: 3,
3   align: center + horizon,
4   table.vline(x:0, stroke: none),
5   table.vline(x:1, stroke: blue),
6   table.vline(x:2),
7   table.header([], [*Col1*] ,
8   [*Col2*], table.hline(stroke: red)),
9   [*Row1*], "cell-0-0", "cell-1-0",
10  [*Row2*], "cell-0-1", "cell-1-1",
11 )

```

	Col1	Col2
Row1	cell-0-0	cell-1-0
Row2	cell-0-1	cell-1-1

```

1 #table(
2   columns: 3,
3   align: center + horizon,
4   stroke: (x:none),
5   table.header([], [*Col1*] ,
6   [*Col2*]),),
7   [*Row1*], "cell-0-0", "cell-1-0",
8   [*Row2*], "cell-0-1", "cell-1-1",
9 )

```

	Col1	Col2
Row1	cell-0-0	cell-1-0
Row2	cell-0-1	cell-1-1

```

1 #table(
2   columns: 3,
3   align: center + horizon,
4   stroke: (y:none),
5   table.header([], [*Col1*] ,
6   [*Col2*]),),
7   [*Row1*], "cell-0-0", "cell-1-0",
8   [*Row2*], "cell-0-1", "cell-1-1",
9 )

```

```

1 #table(
2   columns: 3,
3   align: center + horizon,
4   stroke: (x:none),
5   table.hline(y:0, stroke:none), //
6   remove first line
7   table.hline(y:3, stroke:none), //
8   remove last line
9   table.vline(x:1),
10  table.vline(x:2, start:1, end:2,
11  stroke: red),
12  table.header([], [*Col1*] ,
13  [*Col2*]),),
14  [*Row1*], "cell-0-0", "cell-1-0",
15  [*Row2*], "cell-0-1", "cell-1-1",
16 )

```

	Col1	Col2
Row1	cell-0-0	cell-1-0
Row2	cell-0-1	cell-1-1

<i>c</i>	<i>b</i>	<i>a</i>	<i>cb</i>	<i>ba</i>	<i>y</i>
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	0	0
0	1	1	0	1	0
1	0	0	0	0	0
1	0	1	0	0	1
1	1	0	1	0	1
1	1	1	1	1	1

```
1 #table(  
2   columns: 6,  
3   stroke: none,  
4   align: center+ horizon,  
5   table.vline(x:1, stroke:0.5pt), table.vline(x:2, stroke:0.5pt), table.vline(x:3),  
6   table.vline(x:4, stroke:0.5pt), table.vline(x:5, stroke:0.5pt),  
7   table.header([$c$], [$b$], [$a$], [$c b$], [$b a$], [$y$], table.hline(stroke:  
8   1pt)),  
9   [0], [0], [0], [0], [0], [0], table.hline(stroke: 0.5pt),  
10  [0], [0], [1], [0], [0], [1], table.hline(stroke: 0.5pt),  
11  [0], [1], [0], [0], [0], [0], table.hline(stroke: 0.5pt),  
12  [0], [1], [1], [0], [1], [0], table.hline(stroke: 1pt),  
13  [1], [0], [0], [0], [0], [0], table.hline(stroke: 0.5pt),  
14  [1], [0], [1], [0], [0], [1], table.hline(stroke: 0.5pt),  
15  [1], [1], [0], [1], [0], [1], table.hline(stroke: 0.5pt),  
16  [1], [1], [1], [1], [1], [1],  
17 )
```

4.6.1 Karnaugh Tables

Karnaugh Tables are only available by importing the additional file [kaunaugh.typ](#).

```
1 #import "/00-templates/karnaugh.typ": *
```

$$\begin{array}{cccc}
 & & \overline{c} & \\
 & \overline{b} & & \\
 y & \begin{array}{|c|c|c|c|} \hline 1 & 0 & 1 & 0 \\ \hline 1 & 0 & 1 & 1 \\ \hline \end{array} & a
 \end{array}$$

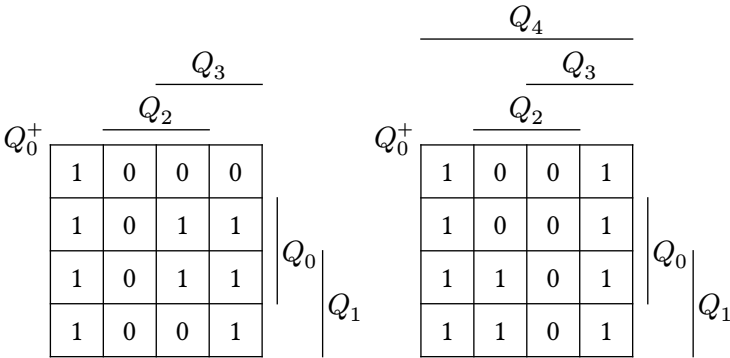
```
1 #karnaugh(content:((1, 0, 1, 0),
2                    (1, 0, 1, 1),))
```

$$\begin{array}{c}
 \begin{array}{c} d \\ \hline c \\ \hline y \end{array}
 \begin{array}{|c|c|c|c|}
 \hline
 1 & 0 & 0 & 0 \\
 \hline
 1 & 0 & 0 & 1 \\
 \hline
 1 & 1 & 0 & 1 \\
 \hline
 1 & 0 & 1 & 1 \\
 \hline
 \end{array}
 \begin{array}{c} \\ \\ a \\ b \end{array}
 \end{array}$$

```
1 #karnaugh(content:((1, 0, 0, 0),
2                    (1, 0, 0, 1),
3                    (1, 1, 0, 1),
4                    (1, 0, 1, 1)),)
```

$$\begin{array}{c}
 d \\
 \hline
 c \\
 \hline
 y \begin{array}{|c|c|c|c|}
 \hline
 1 & 0 & 0 & 0 \\
 \hline
 1 & 0 & 1 & 1 \\
 \hline
 1 & 0 & 1 & 1 \\
 \hline
 1 & 0 & 0 & 1 \\
 \hline
 \end{array}
 \begin{array}{l}
 \left. \vphantom{\begin{array}{|c|c|c|c|} \hline 1 & 0 & 0 & 0 \\ \hline 1 & 0 & 1 & 1 \\ \hline 1 & 0 & 1 & 1 \\ \hline 1 & 0 & 0 & 1 \\ \hline \end{array}} \right| a \\
 b
 \end{array}
 \end{array}$$
$$\begin{array}{c}
 \overline{e} \\
 \overline{d} \\
 \overline{c} \\
 y
 \end{array}
 \begin{array}{|c|c|c|c|}
 \hline
 1 & 0 & 0 & 1 \\
 \hline
 1 & 0 & 0 & 1 \\
 \hline
 1 & 1 & 0 & 1 \\
 \hline
 1 & 1 & 0 & 1 \\
 \hline
 \end{array}
 \begin{array}{l}
 \\
 \\
 a \\
 b
 \end{array}$$

```
1 #karnaugh(content: ((1, 0, 0, 0),
2                      (1, 0, 1, 1),
3                      (1, 0, 1, 1),
4                      (1, 0, 0, 1),
5
6                      (1, 0, 0, 1),
7                      (1, 0, 0, 1),
8                      (1, 1, 0, 1),
9                      (1, 1, 0, 1)))
```



```
1 #karnaugh(inputs: ($Q_0$, $Q_1$, $Q_2$, $Q_3$, $Q_4$),
2   output: [$Q_0^+$],
3   content: ((1, 0, 0, 0),
4             (1, 0, 1, 1),
5             (1, 0, 1, 1),
6             (1, 0, 0, 1),
7
8             (1, 0, 0, 1),
9             (1, 0, 0, 1),
10            (1, 1, 0, 1),
11            (1, 1, 0, 1),))
```

4.7 Icon Boxes

Custom icon boxes are available in the [boxes.typ](#):



1

```
#infobox() ["infobox"]
```



1

```
#ideabox() ["ideabox"]
```



1

```
#warningbox() ["warningbox"]
```



1

```
#importantbox() ["importantbox"]
```



1

```
#firebox() ["firebox"]
```



1

```
#rocketbox() ["rocketbox"]
```



1

```
#todobox() ["todobox"]
```



1

```
#thinkbox() ["thinkbox"]
```



```
1 #helpbox()["helpbox"]
```



```
1 #iconbox(icon: read("/04-resources/
placeholder.svg",encoding:none), linecolor: hei-blue)
["iconbox"]
```

```
1 #iconbox(linecolor: hei-pink)["iconbox without icon"]
```

4.8 Color Boxes

Custom color boxes are available in the [boxes.typ](#):

Exercise

Some text

```
1 #colorbox(title: "Exercise", color: hei-blue)[Some text]
```

Attention

Some text

```
1 #colorbox(title: "Attention", color: hei-pink)[Some text]
```

Consider

Some text

```
1 #slanted-colorbox(title: "Consider", color: hei-green)[Some text]
```

Information

Some text

```
1 #slanted-colorbox(title: "Information", color: hei-orange)[Some text]
```

4.8.1 Todo Box

Custom todo box is available in the [boxes.typ](#):

TODO

This is not finished

```
1 #todo("This is not finished")
```

4.8.2 Option Style

The option style allows to underlight a text depending on the type or state of the document. Within the `/01-settings/metadata.typ` the `option.type` can be set to **draft** or **final** or other types.

Custom option style is available in the [boxes.typ](#):

"This text has an option style and is shown in the case the type is draft"

```
1 #option-style(type:"draft")["This text has an option style and is shown in the case  
the type is draft"]
```

```
1 #option-style(type:"final")["This text has an option style and is shown in the case  
the type is final"]
```

```
1 #option-style(type:"minimal")["This text has an option style and is shown in the  
case the type is minimal"]
```

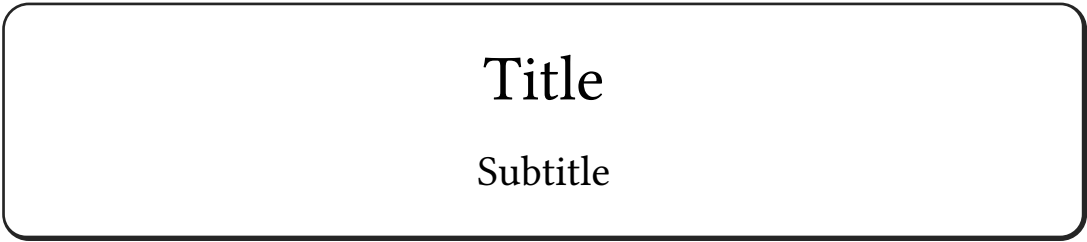
```
1 #option-style(type:"full")["This text has an option style and is shown in the case  
the type is full"]
```

```
1 #option-style(type:"student")["This text has an option style and is shown in the  
case the type is student"]
```

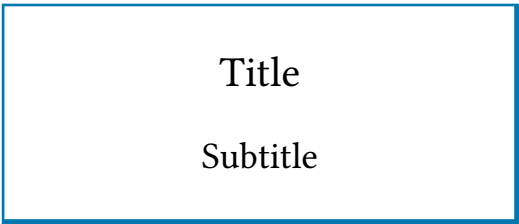
```
1 #option-style(type:"solution")["This text has an option style and is shown in the  
case the type is solution"]
```

4.9 Title Box

Custom title boxes are available in the [boxes.typ](#):



```
1 #titlebox(title: [Title], subtitle: [Subtitle])
```



```
1 #titlebox(width: 50%, radius: 0pt, border: 1pt, linecolor: hei-blue, titlesize: larger, subtitlesize: large, title: [Title], subtitle: [Subtitle])
```



```
1 #titlebox(linecolor: hei-green, titlesize: larger, subtitlesize: large, title: [Title])
```

4.10 Exam Header

Custom exam headers are available in the [boxes.typ](#):



```
#exam-header(nbr-ex: 0, lang: "en")
```

Typst

Grade



```
#exam-header(nbr-ex: 1, lang: "en")
```

Typst

1 Grade

Name:

--	--

(10)

#exam-header(nbr-ex: 2, pts: 10, lang: "en")Typst

Name:

1	2	Grade

(10) (10)

#exam-header(nbr-ex: 3, pts: 10, lang: "en")Typst

Name:

1	2	3	Grade

(10) (10) (10)

#exam-header(nbr-ex: 4, pts: 10, lang: "en")Typst

Name:

1	2	3	4	Grade

(10) (10) (10) (10)

#exam-header(nbr-ex: 5, pts: 10, lang: "en")Typst

Name:

1	2	3	4	5	Grade

(10) (10) (10) (10) (10)

#exam-header(nbr-ex: 6, pts: 10, lang: "en")Typst

Name:

1	2	3	4	5	6	Grade

(10) (10) (10) (10) (10) (10)

#exam-header(nbr-ex: 7, pts: 10, lang: "en")Typst

Name:

1	2	3	4	5	6	7	Grade

(10) (10) (10) (10) (10) (10) (10)

#exam-header(nbr-ex: 8, pts: 10, lang: "en")Typst

1	2	3	4	5	6	7	8	Grade
---	---	---	---	---	---	---	---	-------

Name: _____

(10)	(10)	(10)	(10)	(10)	(10)	(10)	(10)	(10)	(10)

```
#exam-header(nbr-ex: 9, pts: 10, lang: "en")
```

Typst

Name: _____

(10)	(10)	(10)	(10)	(10)	(10)	(10)	(10)	(10)	(10)

```
#exam-header(nbr-ex: 10, pts: 10, lang: "en")
```

Typst

4.11 Exam Reminder

Exam Reminders are only available by importing the additional file [sections.typ](#).

```
1 #import "/00-templates/sections.typ": *
```

Exam Reminder:

You can only use the following items:

- a laptop without internet connection
- a pocketcalculator
- all paper documents you want

It is forbidden to use generative AI.

Good Luck!

```
1 #exam-reminder-did(lang: "en")
```

Prüfungserinnerung:

Sie können nur die folgenden Gegenstände verwenden:

- ein Laptop ohne Internetanschluss
- einen Taschenrechner
- alle Papirdokumente

Es ist verboten, generative KI zu verwenden.

Viel Glück!

```
1 #exam-reminder-did(lang: "de")
```

Rappel d'examen :

Vous ne pouvez utiliser que les éléments suivants :



- un ordinateur portable sans connexion internet
- une calculatrice de poche
- tous les documents papier que vous souhaitez

Il est interdit d'utiliser l'IA générative.

Bonne chance!

```
1 #exam-reminder-did(lang: "fr")
```

Exam Reminder:

You can only use the following items:



- the two-page summary you created.
- a pocketcalculator

In addition, properly comment all high-level and assembler code to explain its purpose and how it fits into the program structure.

Good Luck!

```
1 #exam-reminder-car(lang: "en")
```

Prüfungserinnerung:

Sie können nur die folgenden Elemente verwenden:



- die zweiseitige Zusammenfassung, die Sie erstellt haben.
- einen Taschenrechner

Kommentieren Sie ausserdem den gesamten High-Level- und Assembler-Code ordnungsgemäss aus, um seinen Zweck und seine Einbindung in die Programmstruktur zu erklären.

Viel Glück!

```
1 #exam-reminder-car(lang: "de")
```

Rappel d'examen :

Vous ne pouvez utiliser que les éléments suivants :

- le résumé de deux pages que vous avez créé.
- une calculatrice de poche



Commenter également tout le code de haut niveau et le code assembleur de manière appropriée afin d'expliquer son but et son intégration dans la structure du programme.

Bonne chance!

```
1 #exam-reminder-car(lang: "fr")
```

4.12 Exercise Message

```
1 #import "/00-templates/sections.typ": *
```

Solution vs. Hints:

While not every response provided herein constitutes a comprehensive solution, some serve as helpful hints intended to guide you toward discovering the solution independently. In certain instances, only a portion of the solution is presented.

```
1 #exercises-solution-hints(lang: "en")
```

Lösung vs. Hinweise:

Nicht alle hier gegebenen Antworten sind vollständige Lösungen. Einige dienen lediglich als Hinweise, um Ihnen bei der eigenständigen Lösungsfindung zu helfen. In anderen Fällen wird nur ein Teil der Lösung präsentiert.

```
1 #exercises-solution-hints(lang: "de")
```

**Solution vs. Hints:**

Toutes les réponses fournies ici ne sont pas des solutions complètes. Certaines ne sont que des indices pour vous aider à trouver la solution vous-même. Dans d'autres cas, seule une partie de la solution est fournie.

```
1 #exercices-solution-hints(lang: "fr")
```

4.13 Word/Character Count

With the plugin wordometer is is possible to count the words and characters of a text.

```
1 #import "@preview/wordometer:0.1.5": word-count
```

The following section contains 50 Words and 295 Characters without counting this text.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua quaerat voluptatem. Ut enim aequae doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

```
1 #word-count(total => [
2   #[The following section contains #total.words Words and #total.characters
3   Characters without counting this text.]<no-wc>
4   #lorem(50)
5 ], exclude: <no-wc>)
```



The character count does not include spaces.

5 | References

5.1 Links

Example	Raw
https://tschinz.github.io	<code>https://tschinz.github.io</code>
https://tschinz.github.io	<code>#link("https://tschinz.github.io")</code>
See https://tschinz.github.io	<code>#link("https://tschinz.github.io") [See https://tschinz.github.io]</code>
silvan.zahno@hevs.ch	<code>#link("mailto:silvan.zahno@hevs.com") [silvan.zahno@hevs.ch]</code>
	<code>#link("https://tschinz.github.io") [#image(icon, width:0.5cm)]</code>

Table 8 - Links

5.2 Crossreferences

In the document the following references were added.

```

1 = References <sec:ref>
2 == Links <sec:links>
3 #figure(image("/04-resources/icon.svg", width: 2cm)) <fig:icon>
4 #figure(table(...), kind:table) <tab:links>
5 #figure(align(left, raw(...)) <code-ref>
6 $ sum_{k=1}^n k = (n(n+1)) / 2 $ <math-eq1> #ref(<math-eq1>)

```

Listing 1 - Label inserts

They can be references as follows:

Type	Example	Raw
Section	Section 5	<code>@sec:ref</code>
Subsection	Section 5.1	<code>@sec:links</code>
Figure	Figure 1	<code>@fig:icon</code>
Table	Table 8	<code>@tab:links</code>
Code	Listing 1	<code>@code-ref</code>
Equation	Equation 1	<code>@math-eq1</code>

5.3 External References

Example	Raw
[1]	<code>#cite(label("zahnoDynamicProjectPlanning2023"))</code>
[1, p.7ff]	<code>#cite(<zahnoDynamicProjectPlanning2023>, supplement:[p.7ff])</code>
[1]	<code>@zahnoDynamicProjectPlanning2023</code>

5.4 Glossary

The glossary entries need to be defined in `/03-tail/glossary.typ`.

```

1  #let entry-list = (
2  (
3    key   : "hei",
4    short : "HEI",
5    long  : "Haute École d'Ingénierie",
6  ),
7  (
8    key       : "fpga",
9    short     : "FPGA",
10   plural    : "FPGAs",
11   long      : "Field Programmable Gate Array",
12   longplural : "Field Programmable Gate Arrays",
13   group     : "Technology"
14   description : "A field-programmable gate array (FPGA) is an integrated circuit
15   designed to be configured by a customer or a designer after manufacturing – hence
16   the term 'field-programmable'.",
17 )
18 )

```

For the glossary functions library [Glossarium](#) is used. It can also be used for acronyms.

Example

Raw

Haute École d'Ingénierie
(HEI) and a second time
HEI

```
#gls("hei") and a second time #gls("hei")
```

Haute École d'Ingénierie
(HEI)

```
#gls("hei", first:true)
```

Haute École d'Ingénierie

```
#gls-long("hei")
```

whatever you want

```
#gls("hei", display: "whatever you want"))
```

Field Programmable Gate
Arrays (FPGAs) and a sec-
ond time FPGAs

```
#glspl("fpga") and a second time #glspl("fpga")
```

Field Programmable Gate
Arrays (FPGAs)

```
#glspl("fpga", first: true))
```

Field Programmable Gate
Arrays

```
#glspl-longplural("fpga"))
```

[Infotronics \(IT\)](#) is a specialization of [Systems Engineering \(SYND\)](#) which is part of the [HEI](#). The second time a glossary entry is used the short form will be used: see [IT](#) and [SYND](#). To get the first form back use [Systems Engineering \(SYND\)](#).

6 | Code

There are two ways to include code in Typst: inline and block. For the block code you can use [codly](#) or [code1st](#).

codly retakes the traditional ````lang ```` syntax while **code1st** uses a function taking the traditional syntax `#sourcecode[```lang ```]`.

Typst also provides the **raw** function to include code in the document. The **raw** function can be used for inline and block code.

inline monospaced string

```
1 `inline monospaced string`
```

```
fn main() {println!("Hello world!")}
```

```
1 raw(lang:"rust",
2   "fn main() {println!(\"Hello world!
3   \")}"
4   )
```

```
1-- Test 2: INPUT sX, pp
2opCode <= "INPUT sX, pp  ";
3code    <= "00010";
4cIn     <= '0';
5A       <= "11110000";
6B       <= "00001111";
```

vhdl

```
1 raw(block:true, lang:"vhdl",
2   read("code-example.vhdl"))
```

```
1fn main() {
2  println!("Hello world!")
3}
```

Rust

```
1 ```rust
2 fn main() {
3   println!("Hello world!")
4 }
5 ```
```

```
1 fn main() {
2   println!("Hello world!")
3 }
```

```
1 sourcecode[```rust
2 fn main() {
3   println!("Hello world!")
4 }
5 ```]
```

```
1fn main() {
2  println!("Hello world!")
3}
```

Rust

```
1 #figure(
2   align(left,
3     ```rust
4     fn main() {
5       println!("Hello world!")
6     }
7     ```
8   ),
9   caption: [Rust Code],
10 )
```

Listing 2 - Rust Code

7 | Math Equations

Inline math

Let a and b , and c be the side of a right-angled triangle.

Let a and b , and c be the side of a right-angled triangle.

$$\sum_{k=1}^n k = \frac{n(n+1)}{2}$$

$\sum_{k=1}^n k = (n(n+1)) / 2$,

Fullline math

$$a^2 + b^2 = c^2 \tag{1}$$

$a^2 + b^2 = c^2$ `<math-eq1>`

Math with caption

$$\sum_{k=1}^n k = \frac{n(n+1)}{2} \tag{2}$$

Equation 8 - Some proof

```
#figure(
  $ \sum_{k=1}^n k = (n(n+1)) / 2 $,
  caption: [Some proof]
)
```

7.1 Align

Formula

$$\begin{aligned} a_1 &= b_1 + c_1 = z_1 \\ a_2 &= b_2 + c_2 - d_2 + e_2 = z_1 \end{aligned}$$

Raw

(3)
$$\begin{aligned} a_1 &= b_1 + c_1 = z_1 \\ a_2 &= b_2 + c_2 - d_2 + e_2 = z_1 \end{aligned}$$

$$\begin{aligned} a_1 &= b_1 + c_1 &= z_1 \\ a_2 &= b_2 + c_2 - d_2 + e_2 = z_1 \end{aligned}$$

(4)
$$\begin{aligned} a_1 &\&= b_1 + c_1 \&= z_1 \\ a_2 &\&= b_2 + c_2 - d_2 + e_2 \&= z_1 \end{aligned}$$

7.2 Symbols

This is an incomplete list for all symbols goto [here](#)

Outside of the $\$$ math environment the symbols can be accessed with .

7.2.1 Accents

Sym	Raw	Sym	Raw	Sym	Raw
$\grave{x}$	<code>\$grave(x)\$</code>	$\acute{x}$	<code>\$acute(x)\$</code>	$\hat{x}$	<code>\$hat(x)\$</code>
$\tilde{x}$	<code>\$tilde(x)\$</code>	$\breve{x}$	<code>\$breve(x)\$</code>	$\dot{x}$	<code>\$dot(x)\$</code>
$\ddot{x}$	<code>\$dot.double(x)\$</code>	$\ddot{x}$	<code>\$dot.triple(x)\$</code>	$\ddot{x}$	<code>\$dot.quad(x)\$</code>
$\ddot{x}$	<code>\$diaer(x)\$</code>	$\mathring{x}$	<code>\$circle(x)\$</code>	$\ddot{x}$	<code>\$acute.double(x)\$</code>
$\tilde{x}$	<code>\$caron(x)\$</code>	$\vec{x}$	<code>\$arrow(x)\$</code>	$\vec{x}$	<code>\$arrow.l(x)\$</code>
$\cancel{x}$	<code>\$cancel(x)\$</code>	$\bar{x}$	<code>\$macron(x)\$</code>	$\overline{xyz}$	<code>\$overline(xyz)\$</code>
$\overline{xyz}$	<code>\$overline(xyz)\$</code>	$\underbrace{xyz}$	<code>\$underbrace(xyz)\$</code>	$\overbrace{xyz}$	<code>\$overbrace(xyz)\$</code>
$\underbrace{xyz}$	<code>\$underbracket(xyz)\$</code>	$\overbrace{xyz}$	<code>\$overbracket(xyz)\$</code>	$\overbrace{xyz}$	<code>\$overbracket(xyz)\$</code>

7.2.2 Equals & Operators

Sym	Raw	Sym	Raw	Sym	Raw
$=$	<code>\$=\$</code>	$=$	<code>\$eq\$</code>	$\neq$	<code>\$eq.not\$</code>
$\neq$	<code>\$!= \$</code>	$\equiv$	<code>\$equiv\$</code>	$\neq$	<code>\$equiv.not\$</code>
$\simeq$	<code>\$tilde.eq\$</code>	$\not\equiv$	<code>\$tilde.eq.not\$</code>	$\geq$	<code>\$gt.eq\$</code>
$\nlessgtr$	<code>\$gt.eq.not\$</code>	$\leq$	<code>\$lt.eq\$</code>	$\nlessgtr$	<code>\$lt.eq.not\$</code>
$\approx$	<code>\$approx\$</code>	$\cong$	<code>\$approx.eq\$</code>	$\not\approx$	<code>\$approx.not\$</code>
$:$	<code>\$colon\$</code>	$:=$	<code>\$colon.eq\$</code>	$=:$	<code>\$eq.colon\$</code>
$::=$	<code>\$colon.double.eq\$</code>	$+$	<code>\$+\$</code>	$+$	<code>\$plus\$</code>
$\pm$	<code>\$plus.minus\$</code>	$\oplus$	<code>\$plus.o\$</code>	$-$	<code>\$-\$</code>
$-$	<code>\$minus\$</code>	$\mp$	<code>\$minus.plus\$</code>	$\ominus$	<code>\$minus.o\$</code>

7.2.3 Scripts

Sym	Raw	Sym	Raw	Sym	Raw
x_1	<code>\$x_1\$</code>	x_{12}	<code>\$x_(12)\$</code>	x_1	<code>\$scripts(x)_1\$</code>
x_1	<code>\$x_1\$</code>	x_{12}	<code>\$x_(12)\$</code>	x_1	<code>\$scripts(x)_1\$</code>
x_1^2	<code>\$x_1^2\$</code>	x_{12}^{34}	<code>\$x_(12)^(34)\$</code>	x_1^2	<code>\$scripts(x)_1^2\$</code>
$\frac{x}{1}$	<code>\$x_1^2\$</code>	x_{12}^{34}	<code>\$x_(12)^(34)\$</code>	x_1^2	<code>\$scripts(x)_1^2\$</code>

7.2.4 Special Elements

Sym	Raw	Sym	Raw
$\binom{n}{k}$	(5) <code>\$ binom(n, k) \$</code>	$\begin{bmatrix} 1 \\ 2 \end{bmatrix}$	(6) <code>\$ vec(1, 2, delim: "[") \$</code>
$\left\lfloor \frac{1}{2} \right\rfloor$	(7) <code>\$ round(1, 2) \$</code>	$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$	(8) <code>\$ mat(1,2; 3,4) \$</code>

$$\begin{pmatrix} 1 & 2 & \dots & 10 \\ 2 & 2 & \dots & 10 \\ \vdots & \vdots & \ddots & \vdots \\ 10 & 10 & \dots & 10 \end{pmatrix} \quad (9)$$

```

1 $ mat(
2   1, 2, ..., 10;
3   2, 2, ..., 10;
4   dots.v,
5   dots.v,
6   dots.down,
7   dots.v;
8   10, 10, ...,
9   10;
10  ) $

```

$$\sum a_k \quad (10) \quad \$ \text{sum } \mathbf{a_k} \$$$

$$\sum_{k=0}^n a_k \quad (11) \quad \$ \text{sum_}(k=0)^n \mathbf{a_k} \$$$

$$\sum_{k=0}^n a_k \quad (12) \quad \$ \text{scripts}(\text{sum})_ (k=0)^n \mathbf{a_k} \$$$

$$\sqrt[3]{x} \quad (13) \quad \$ \text{root}(3, \mathbf{x}) \$$$

$$f(x, y) := \begin{cases} 1 & \text{if } \frac{x \cdot y}{2} \leq 0 \\ 2 & \text{if } x \text{ is even} \\ 3 & \text{if } x \in \mathbb{N} \\ 4 & \text{else} \end{cases} \quad (14)$$

```

1 $ f(x, y) := cases(
2   1 "if" (x dot y)/2 <= 0,
3   2 "if" x "is even",
4   3 "if" x in NN,
5   4 "else",
6 ) $

```

$$\frac{1}{2} \quad (15) \quad \$ \mathbf{1/2} \$$$

$$\frac{1}{2} \quad (16) \quad \$ \text{frac}(\mathbf{1}, \mathbf{2}) \$$$

$$\frac{x+1}{x+2} \quad (17) \quad \$ (\mathbf{x+1})/(\mathbf{x+2}) \$$$

$$\frac{(x+1)}{(x+2)} \quad (18) \quad \$ ((\mathbf{x+1}))/((\mathbf{x+2})) \$$$

$$\prod \quad (19) \quad \$ \text{product} \$$$

$$n! = \prod_{k=1}^n k \quad (20) \quad \$ \mathbf{n!} = \text{product_}(k=1)^n \mathbf{k} \$$$

$$n! = \prod_{k=1}^n k \quad (21) \quad \$ \mathbf{n!} = \text{scripts}(\text{product})_ (k=1)^n \mathbf{k} \$$$

$$\int \quad (22) \quad \$ \text{integral} \$$$

$$\int_a^b f(x) \quad (23) \quad \$ \text{integral} \$$$

7.2.5 Alphabeth

Sym

$\alpha\beta\gamma\delta\epsilon\zeta\eta\theta\iota\kappa\lambda\mu\nu\xi\omicron\pi\rho\sigma\tau\upsilon\varphi\chi\psi\omega$

ΑΒΓΔΕΖΗΘΙΚΑΜΝΞΟΠΡΣΤΥΦΧΨΩ

Raw

α β γ δ ϵ ζ η θ ι κ λ μ ν ξ $\omicron$ π ρ σ τ υ φ χ ψ ω

α β γ δ ϵ ζ η θ ι κ λ μ ν ξ $\omicron$ π ρ σ τ υ φ χ ψ ω

ABCDEFGHIJKLMNOPQRSTUVWXYZ

\$AA BB CC DD EE FF GG HH II JJ KK LL MM
NN OO PP QQ RR SS TT UU VV WW XX YY ZZ\$

7.2.6 Logical

Sym	Raw	Sym	Raw	Sym	Raw
$\wedge$	<code>\$and\$</code>	$\bigwedge$	<code>\$and.big\$</code>	$\&$	<code>\$amp\$</code>
$\vee$	<code>\$or\$</code>	$\bigvee$	<code>\$bar.v\$</code>	$*$	<code>\$ast.op\$</code>
$*$	<code>\$ast.basic\$</code>	$*$	<code>\$ast.low\$</code>	$\oplus$	<code>\$plus.o\$</code>
$\oplus$	<code>\$plus.o.big\$</code>				

7.2.7 Operators

Sym	Raw	Sym	Raw	Sym	Raw
$\sin x$	<code>\$sin x\$</code>	$\cos x$	<code>\$cos x\$</code>	$\tan x$	<code>\$tan x\$</code>
$\arcsin x$	<code>\$arcsin x\$</code>	$\arccos x$	<code>\$arccos x\$</code>	$\arctan x$	<code>\$arctan x\$</code>
$\sinh x$	<code>\$sinh x\$</code>	$\cosh x$	<code>\$cosh x\$</code>	$\tanh x$	<code>\$tanh x\$</code>
$\arg x$	<code>\$arg x\$</code>	$\csc x$	<code>\$csc x\$</code>	$\deg x$	<code>\$deg x\$</code>
$\det x$	<code>\$det x\$</code>	$\dim x$	<code>\$dim x\$</code>	$\exp x$	<code>\$exp x\$</code>
$\bmod x$	<code>\$mod x\$</code>	$\inf x$	<code>\$inf x\$</code>	$\log x$	<code>\$log x\$</code>
$\lim x$	<code>\$lim x\$</code>	$\liminf x$	<code>\$liminf x\$</code>	$\limsup x$	<code>\$limsup x\$</code>
$\min x$	<code>\$min x\$</code>	$\max x$	<code>\$max x\$</code>	$\sup x$	<code>\$sup x\$</code>

7.2.8 Arrows

SymRaw	SymRaw	SymRaw	
	Arrows right		
$\rightarrow$	<code>\$arrow\$</code>	$\mapsto$	<code>\$arrow.bar\$</code>
$\mapsto$	<code>\$arrow.bar.long\$</code>	$\Rightarrow$	<code>\$arrow.double.long\$</code>
$\mapsto$	<code>\$arrow.double.bar\$</code>	$\Rightarrow$	<code>\$arrow.double.bar.long\$</code>
$\Rightarrow$	<code>\$arrow.stroked\$</code>	$\Rightarrow$	<code>\$arrow.quad\$</code>
$\hookrightarrow$	<code>\$arrow.curve\$</code>	$\dashrightarrow$	<code>\$arrow.dashed\$</code>
		$\looparrowright$	<code>\$arrow.loop\$</code>
	Arrows left		
$\leftarrow$	<code>\$arrow.l\$</code>	$\leftarrow$	<code>\$arrow.l.long\$</code>
$\leftarrow$	<code>\$arrow.l.bar.long\$</code>	$\leftarrow$	<code>\$arrow.l.double\$</code>
$\leftarrow$	<code>\$arrow.l.double.bar\$</code>	$\leftarrow$	<code>\$arrow.l.double.bar.long\$</code>
$\leftarrow$	<code>\$arrow.l.stroked\$</code>	$\leftarrow$	<code>\$arrow.l.quad\$</code>
$\hookleftarrow$	<code>\$arrow.l.curve\$</code>	$\dashleftarrow$	<code>\$arrow.l.dashed\$</code>
		$\looparrowleft$	<code>\$arrow.l.loop\$</code>
	Double Arrows Left Right		
$\leftrightarrow$	<code>\$arrow.l.r\$</code>	$\nleftrightarrow$	<code>\$arrow.l.r.not\$</code>
		$\longleftrightarrow$	<code>\$arrow.l.r.long\$</code>

$\Leftrightarrow$ `$arrow.l.r.double$` $\Leftrightarrow$ `$arrow.l.r.double.long$` $\nleftrightarrow$ `$arrow.l.r.double.not$`
 $\rightleftarrows$ `$arrow.l.r.stroked$` $\longleftrightarrow$ `$arrow.l.r.filled$` $\curvearrowright$ `$arrow.l.r.wave$`

Arrows Top

$\uparrow$ `$arrow.t$` $\Uparrow$ `$arrow.t.bar$` $\Uparrow$ `$arrow.t.double$`
 $\Uparrow$ `$arrow.t.triple$` $\Uparrow$ `$arrow.t.quad$` $\Uparrow$ `$arrow.t.stroked$`
 $\Uparrow$ `$arrow.t.filled$` $\Uparrow$ `$arrow.t.dashed$` $\nearrow$ `$arrow.t.curve$`

Arrows Bottom

$\downarrow$ `$arrow.b$` $\Downarrow$ `$arrow.b.bar$` $\Downarrow$ `$arrow.b.double$`
 $\Downarrow$ `$arrow.b.triple$` $\Downarrow$ `$arrow.b.quad$` $\Downarrow$ `$arrow.b.stroked$`
 $\Downarrow$ `$arrow.b.filled$` $\Downarrow$ `$arrow.b.dashed$` $\searrow$ `$arrow.b.curve$`

Double Arrows Top Bottom

$\Updownarrow$ `$arrow.t.b$` $\Updownarrow$ `$arrow.t.b.double$` $\Updownarrow$ `$arrow.t.b.stroked$`
 $\Updownarrow$ `$arrow.t.b.filled$`

Arrows Diagonal Top Right

$\nearrow$ `$arrow.tr$` $\nearrow$ `$arrow.tr.double$` $\nearrow$ `$arrow.tr.stroked$`
 $\nearrow$ `$arrow.tr.filled$` $\nearrow$ `$arrow.tr.hook$`

Arrows Diagonal Bottom Right

$\searrow$ `$arrow.br$` $\searrow$ `$arrow.br.double$` $\searrow$ `$arrow.br.stroked$`
 $\searrow$ `$arrow.br.filled$` $\searrow$ `$arrow.br.hook$`

Arrows Diagonal Bottom Left

$\swarrow$ `$arrow.bl$` $\swarrow$ `$arrow.bl.double$` $\swarrow$ `$arrow.bl.stroked$`
 $\swarrow$ `$arrow.bl.filled$` $\swarrow$ `$arrow.bl.hook$`

Arrows Diagonal Top Left

$\nwarrow$ `$arrow.tl$` $\nwarrow$ `$arrow.tl.double$` $\nwarrow$ `$arrow.tl.stroked$`
 $\nwarrow$ `$arrow.tl.filled$` $\nwarrow$ `$arrow.tl.hook$`

Double Arrows Diagonal

$\Updownarrow$ `$arrow.tl.br$` $\Updownarrow$ `$arrow.tr.bl$`

Other Arrows

$\circlearrowright$ `$arrow.cw$` $\circlearrowright$ `$arrow.cw.half$` $\circlearrowleft$ `$arrow.ccw$`
 $\circlearrowleft$ `$arrow.ccw.half$`

7.2.9 Angles

Sym	Raw	Sym	Raw	Sym	Raw
$\angle$	<code>\$angle\$</code>	$\sphericalangle$	<code>\$angle.rev\$</code>	$\sphericalangle$	<code>\$angle.acute\$</code>
$\sphericalangle$	<code>\$angle.acute\$</code>	$\sphericalangle$	<code>\$angle.arc\$</code>	$\sphericalangle$	<code>\$angle.arc.rev\$</code>
$\langle$	<code>\$chevron.l.l\$</code>	$\langle$	<code>\$chevron.l.l.double\$</code>	$\rangle$	<code>\$chevron.r.r\$</code>
$\rangle$	<code>\$chevron.r.r.double\$</code>	$\angle$	<code>\$angle.right\$</code>	$\sphericalangle$	<code>\$angle.right.rev\$</code>

	<code>\$angle.right.arc\$</code>		<code>\$angle.right.dot\$</code>		<code>\$angle.right.square\$</code>
	<code>\$angle.spheric\$</code>		<code>\$angle.spheric.rev\$</code>		<code>\$angle.spheric.t\$</code>

7.2.10 Cool Symbols

Sym	Raw	Sym	Raw	Sym	Raw
@	<code>\$at\$</code>	%	<code>\$co\$</code>	©	<code>\$copyright\$</code>
©	<code>\$copyright.sound\$</code>	°C	<code>\$degree C\$</code>	€	<code>\$euro\$</code>
\$	<code>\$dollar\$</code>	£	<code>\$pound\$</code>	₩	<code>\$won\$</code>
¥	<code>\$yen\$</code>	฿	<code>\$bitcoin\$</code>	°F	<code>\$degree F\$</code>
!	<code>\$excl\$</code>	¡	<code>\$excl.inv\$</code>	!!	<code>\$excl.double\$</code>
!?	<code>\$excl.quest\$</code>	↯	<code>\$arrow.zigzag\$</code>	⊗	<code>\$ast.op.o\$</code>
⋆	<code>\$ast.triple\$</code>	χ	<code>\$chi\$</code>	☐	<code>\$floral\$</code>
⌘	<code>\$maltese\$</code>	¶	<code>\$pilcrow\$</code>	ℏ	<code>\$planck\$</code>
♣	<code>\$suit.club\$</code>	♦	<code>\$suit.diamond\$</code>	♥	<code>\$suit.heart\$</code>
♠	<code>\$suit.spade\$</code>	△	<code>\$triangle.stroked.nested2\$</code>		

7.2.11 Style

Sym	Raw	Sym	Raw
ABC123	<code>\$sans(A B C 1 2 3)\$</code>	ℳ℔℔123	<code>\$frak(A B C 1 2 3)\$</code>
ABC123	<code>\$mono(A B C 1 2 3)\$</code>	ABC123	<code>\$bb(A B C 1 2 3)\$</code>
ABC123	<code>\$cal(A B C 1 2 3)\$</code>		

Sym	Raw
-----	-----

$\sum_{i \in \mathbb{N}} 1 + i$

```
1 #show math.equation: set text(font: "Cambria Math")
2 $sum_(i in NN) 1 + i$,
```

8 | Emoji Symbols

This is an incomplete list for all emoji goto [here](#)

If the emoji module is imported the `#emoji` can be removed

```
1 #import emoji: *
```

Sym Raw

Sym Raw

Smileys

Sym Raw

	<code>#emoji.face</code>		<code>#emoji.baby</code>		<code>#emoji.alien</code>
	<code>#emoji.alien.monster</code>		<code>#emoji.alien.monster</code>		<code>#emoji.eye</code>
	<code>#emoji.eyes</code>		<code>#emoji.face.grin</code>		<code>#emoji.face.angry</code>
	<code>#emoji.face.angry.red</code>		<code>#emoji.face.anguish</code>		<code>#emoji.face.astonish</code>
	<code>#emoji.face.bandage</code>		<code>#emoji.face.beam</code>		<code>#emoji.face.concern</code>
	<code>#emoji.face.cool</code>		<code>#emoji.face.cover</code>		<code>#emoji.face.down</code>
	<code>#emoji.face.down.sweat</code>		<code>#emoji.face.drool</code>		<code>#emoji.face.explode</code>
	<code>#emoji.face.eyeroll</code>		<code>#emoji.face.friendly</code>		<code>#emoji.face.fear</code>
	<code>#emoji.face.fear.sweat</code>		<code>#emoji.face.fever</code>		<code>#emoji.face.flush</code>
	<code>#emoji.face.halo</code>		<code>#emoji.face.happy</code>		<code>#emoji.face.heart</code>
	<code>#emoji.face.hearts</code>		<code>#emoji.face.heat</code>		<code>#emoji.face.kiss</code>
	<code>#emoji.face.kiss.heart</code>		<code>#emoji.face.kiss.blush</code>		<code>#emoji.face.joy</code>
	<code>#emoji.face.lie</code>		<code>#emoji.face.mask</code>		<code>#emoji.face.meh</code>
	<code>#emoji.face.melt</code>		<code>#emoji.face.neutral</code>		<code>#emoji.face.party</code>
	<code>#emoji.face.peek</code>		<code>#emoji.face.plead</code>		<code>#emoji.face.relief</code>
	<code>#emoji.face.vomit</code>		<code>#emoji.face.unhappy</code>		<code>#emoji.face.wow</code>
	<code>#emoji.face.yawn</code>		<code>#emoji.face.wink</code>		<code>#emoji.face.zip</code>
	<code>#emoji.finger.l</code>		<code>#emoji.finger.t</code>		<code>#emoji.finger.b</code>
	<code>#emoji.finger.m</code>		<code>#emoji.finger.front</code>		<code>#emoji.fingers.cross</code>
	<code>#emoji.fingers.pinch</code>		<code>#emoji.fingers.snap</code>		<code>#emoji.fist.front</code>
	<code>#emoji.fist.r</code>		<code>#emoji.fist.l</code>		<code>#emoji.fist.raised</code>
	<code>#emoji.glasses</code>		<code>#emoji.glasses.sun</code>		<code>#emoji.hand.raised</code>
	<code>#emoji.hand.r</code>		<code>#emoji.hand.l</code>		<code>#emoji.hand.t</code>

Sym Raw

 #emoji.hand.b
 #emoji.hand.love
 #emoji.hand.pinch
 #emoji.hand.write
 #emoji.hands.heart

Sym Raw

 #emoji.hand.ok
 #emoji.hand.part
 #emoji.hand.rock
 #emoji.hands.folded
 #emoji.hands.shake

Sym Raw

 #emoji.hand.call
 #emoji.hand.peace
 #emoji.hand.wave
 #emoji.hands.clap
 #emoji.hand.splay

Drinks

Sym Raw

 #emoji.beer
 #emoji.coffee
 #emoji.glass.clink
 #emoji.teacup

Sym Raw

 #emoji.beer.clink
 #emoji.cup.straw
 #emoji.glass.milk
 #emoji.mate

Sym Raw

 #emoji.cocktail.tropical
 #emoji.glass.tumbler
 #emoji.teapot
 #emoji.soup

Animals

Sym Raw

 #emoji.ant
 #emoji.beetle
 #emoji.beetle.lady
 #emoji.bison
 #emoji.butterfly
 #emoji.cat
 #emoji.chipmunk
 #emoji.crocodile
 #emoji.dodo
 #emoji.duck
 #emoji.fairy
 #emoji.flamingo
 #emoji.giraffe

Sym Raw

 #emoji.badger
 #emoji.bee
 #emoji.beaver
 #emoji.boar
 #emoji.camel
 #emoji.chicken
 #emoji.cockroach
 #emoji.dino.pod
 #emoji.dog
 #emoji.egg
 #emoji.feather
 #emoji.fly
 #emoji.hedgehog

Sym Raw

 #emoji.bat
 #emoji.beetle
 #emoji.bear
 #emoji.bug
 #emoji.camel.dromedar
 #emoji.chicken.male
 #emoji.cricket
 #emoji.dino.rex
 #emoji.dolphin
 #emoji.elephant
 #emoji.fish
 #emoji.goat
 #emoji.hippo

Flowers

Sym Raw

 #emoji.fleur
 #emoji.flower.pink

Sym Raw

 #emoji.flower.hibiscus
 #emoji.flower.rose

Sym Raw

 #emoji.flower.lotus
 #emoji.flower.sun

Sym Raw

 #emoji.flower.wilted

Sym Raw

 #emoji.flower.yellow

Sym Raw

 #emoji.plant

Food

Sym Raw

 #emoji.avocado #emoji.broccoli #emoji.carrot #emoji.cucumber

Sym Raw

 #emoji.aubergine #emoji.burger #emoji.chocolate #emoji.fries

Sym Raw

 #emoji.banana #emoji.cake.slice #emoji.corn #emoji.honey

Objects

Sym Raw

 #emoji.bin #emoji.books #emoji.bell #emoji.broom #emoji.cabinet.file #emoji.compass

Sym Raw

 #emoji.bomb #emoji.book #emoji.bell.not #emoji.brush #emoji.chair #emoji.comet

Sym Raw

 #emoji.bone #emoji.book.open #emoji.briefcase #emoji.bubbles #emoji.coffin #emoji.hook

Sport

Sym Raw

 #emoji.baseball #emoji.chess

Sym Raw

 #emoji.billiards #emoji.climbing

Sym Raw

 #emoji.boxing #emoji.football

Vehicles

Sym Raw

 #emoji.airplane #emoji.boat.sail #emoji.car #emoji.ship

Sym Raw

 #emoji.bike #emoji.boat.speed #emoji.car.front #emoji.motorcycle

Sym Raw

 #emoji.boat #emoji.cablecar #emoji.car.racing #emoji.wheelchair.motor

Building

Sym Raw

 #emoji.castle.eu #emoji.factory #emoji.house

Sym Raw

 #emoji.castle.jp #emoji.hospital #emoji.museum

Sym Raw

 #emoji.circus #emoji.office #emoji.crane

Weather

Sym Raw

Sym Raw

Sym Raw

Sym Raw

	<code>#emoji.cloud</code>
	<code>#emoji.cloud.snow</code>
	<code>#emoji.cloud.sun.hidden</code>
	<code>#emoji.drop</code>

Sym Raw

	<code>#emoji.cloud.dust</code>
	<code>#emoji.cloud.storm</code>
	<code>#emoji.cloud.sun.rain</code>
	<code>#emoji.drops</code>

Sym Raw

	<code>#emoji.cloud.rain</code>
	<code>#emoji.cloud.sun</code>
	<code>#emoji.cloud.thunder</code>
	<code>#emoji.sun</code>

Special

Sym Raw

✓	<code>#emoji.checkmark.heavy</code>
	<code>#emoji.circle.white</code>
✗	<code>#emoji.crossmark</code>
	<code>#emoji.explosion</code>
	<code>#emoji.flag.white</code>
	<code>#emoji.flag.red</code>
	<code>#emoji.globe.eu.af</code>
	<code>#emoji.notes</code>

Sym Raw

	<code>#emoji.checkmark.box</code>
	<code>#emoji.circle.stroked</code>
	<code>#emoji.crossmark.box</code>
	<code>#emoji.fire</code>
	<code>#emoji.flag.black</code>
	<code>#emoji.globe.am</code>
	<code>#emoji.globe.meridian</code>
	<code>#emoji.notes.triple</code>

Sym Raw

	<code>#emoji.circle.black</code>
©	<code>#emoji.copyright</code>
!	<code>#emoji.excl</code>
	<code>#emoji.firecracker</code>
	<code>#emoji.flag.goal</code>
	<code>#emoji.globe.as.au</code>
#	<code>#emoji.hash</code>
*	<code>#emoji.ast</code>

Technology

Sym Raw

	<code>#emoji.battery</code>
	<code>#emoji.brightness.low</code>
	<code>#emoji.calendar</code>
	<code>#emoji.cassette</code>
	<code>#emoji.clamp</code>
	<code>#emoji.clock.alarm</code>
	<code>#emoji.controller</code>
	<code>#emoji.email</code>
	<code>#emoji.folder</code>
	<code>#emoji.hammer</code>
	<code>#emoji.laptop</code>

Sym Raw

	<code>#emoji.battery.low</code>
	<code>#emoji.bubble.speech.l</code>
	<code>#emoji.camera</code>
	<code>#emoji.clip</code>
	<code>#emoji.clock.one</code>
	<code>#emoji.computer</code>
	<code>#emoji.crab</code>
	<code>#emoji.floppy</code>
	<code>#emoji.folder.open</code>
	<code>#emoji.hammer.wrench</code>

Sym Raw

	<code>#emoji.brightness.high</code>
	<code>#emoji.bubble.speech.r</code>
	<code>#emoji.camera.movie</code>
	<code>#emoji.clipboard</code>
	<code>#emoji.clock.two</code>
	<code>#emoji.computermouse</code>
	<code>#emoji.disc.cd</code>
	<code>#emoji.flashlight</code>
	<code>#emoji.gear</code>
	<code>#emoji.headphone</code>

Glossary

FPGA – Field Programmable Gate Array: A field-programmable gate array (FPGA) is an integrated circuit designed to be configured by a customer or a designer after manufacturing – hence the term 'field-programmable'. [31](#), [31](#), [31](#)

HEI – Haute École d'Ingénierie [31](#), [31](#), [31](#), [31](#), [31](#)

IT – Infotronics [31](#), [31](#)

SYND – Systems Engineering [31](#), [31](#), [31](#)

Bibliography

- [1] S. Zahno *et al.*, “Dynamic Project Planning with Digital Twin,” *Frontiers in Manufacturing Technology*, vol. 3, May 2023, doi: [10.3389/fmtec.2023.1009633](https://doi.org/10.3389/fmtec.2023.1009633).